

《移动终端软件开发技术》

Mobile Application Development

7. Android后台服务

谢坤
计算机学院（国家示范性软件学院）

1 Service简介

1.1 Service简介

- Service:
 - Android中实现程序后台运行的解决方案
 - 适合执行不需要和用户交互而且还要求长期运行的任务
 - 运行不依赖用户界面
 - 运行依赖于所在的应用程序进程
 - 不自动开启线程，所有的代码默认运行在主线程当中

1.1 Service简介

- Service的优势：
 - 无用户界面，有利于降低系统资源消耗
 - 进程优先级居中
 - 在系统资源恢复后也可自动恢复运行状态
 - 可用于进程间通信（Inter Process Communication, IPC）

1.2 Service使用方式

- Service使用方式：
 - 启动方式
 - 主要用于启动一个服务执行后台任务，不进行通信
 - 绑定方式
 - 主要用于服务组件的调用，调用者可以与组件进行通信
 - 同时使用启动方式与绑定方式

1.2 Service使用方式

- 启动方式：
 - 通过调用Context.startService() 启动
 - 通过调用Context.stopService() 或服务.stopSelf() 停止
 - 由其它的组件启动，由其它组件或自身停止
 - 启动Service的组件不能获取Service的对象实例，无法调用Service中的任何函数，也不能获取Service中的任何状态和数据信息
 - 能够以启动方式使用的Service，需具备自我管理的能力。

1.2 Service使用方式

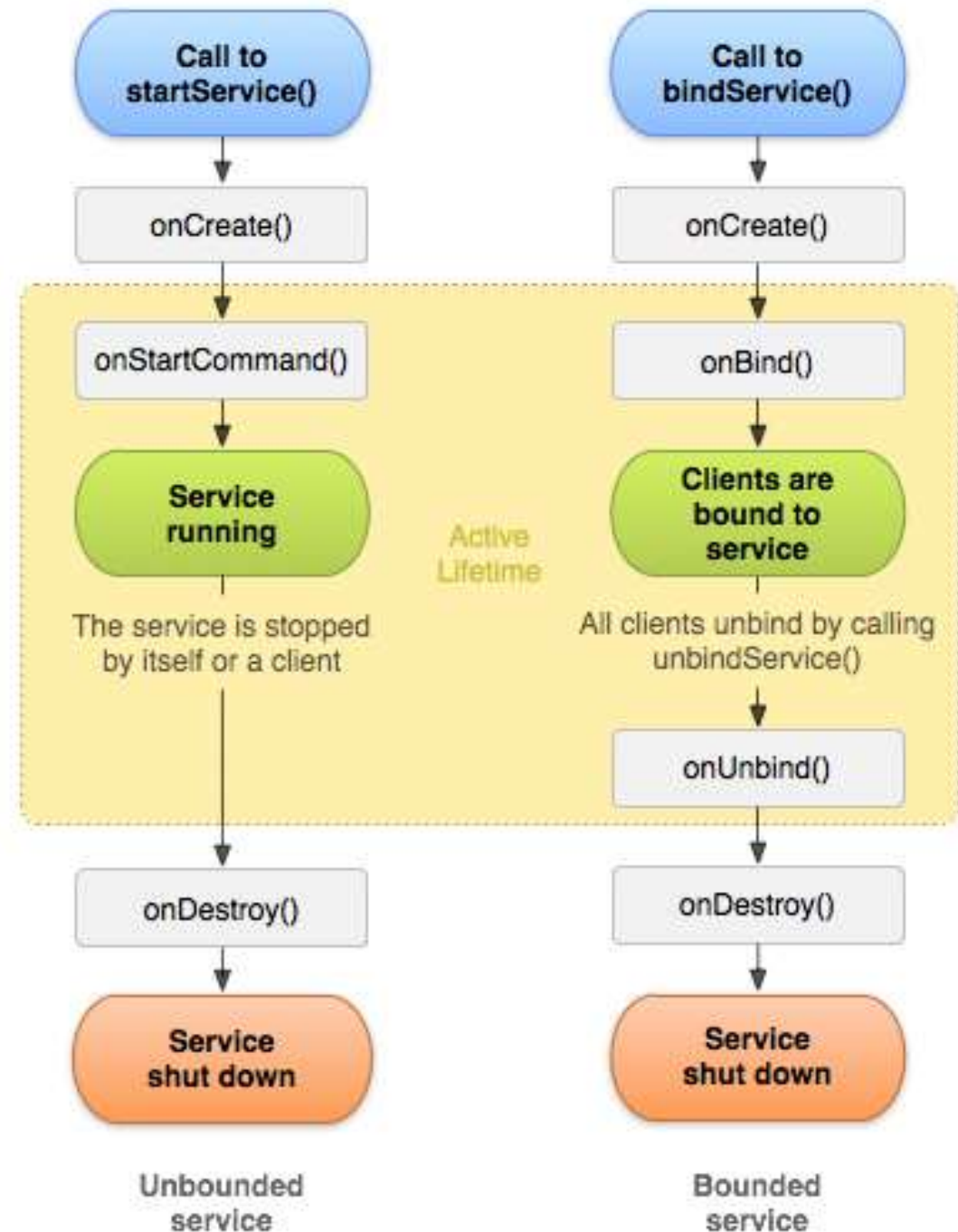
- 绑定方式：
 - 通过服务链接（ServiceConnection）实现
 - 可以获取Service对象实例
 - 可以调用Service中实现的函数
 - 可以获取Service的状态和数据信息
 - 通过Context.bindService() 建立服务链接
 - Service若未启动，此函数会自动启动Service
 - 通过Context.unbindService() 停止服务链接
 - 同一个Service可绑定多个服务链接
 - 同时为多个不同的组件提供服务

1.2 Service使用方式

- 启动方式和绑定方式的结合：
 - 通过Context.startService() 启动
 - 通过Context.bindService() 获取服务链接和服务对象实例
 - 通过调用实例中的函数管控服务，并保存相关信息
 - 此时仅调用Context.stopService() 不能直接停止Service，在所有的服务链接关闭后，Service才能真正停止
 - 此时仅调用unbindService() 停止所有服务链接也不能直接停止Service，在调用Context.stopService() 方法后，Service才能真正停止

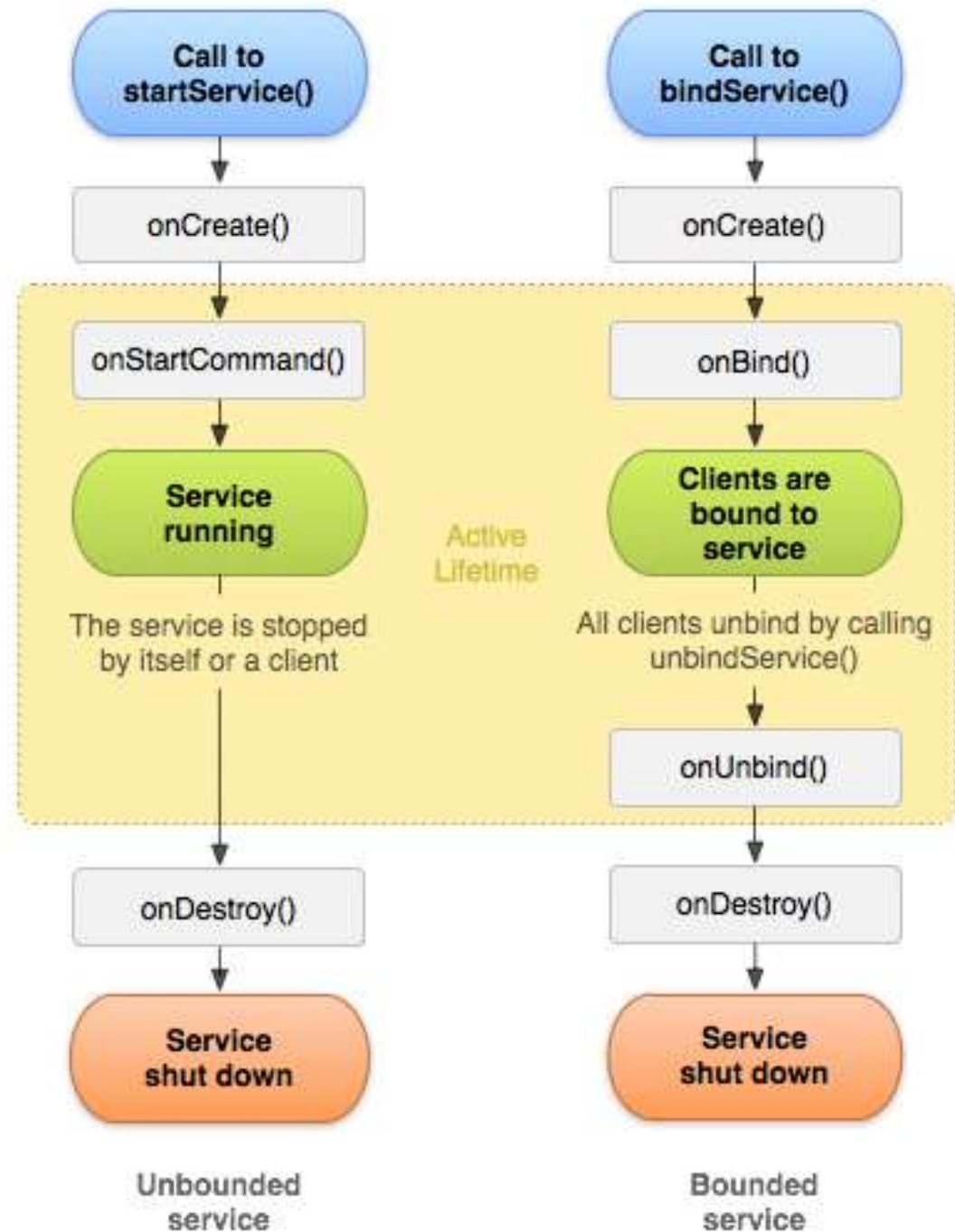
1.3 Service生命周期

- `onCreate()`: 首次创建服务时, 系统将调用此方法。如果服务已在运行, 则不会调用此方法, 该方法只调用一次。
- `onStartCommand()`: 当另一个组件通过调用`startService()`请求启动服务时, 系统将调用此方法。
- `onDestroy()`: 当服务不再使用且将被销毁时, 系统将调用此方法。



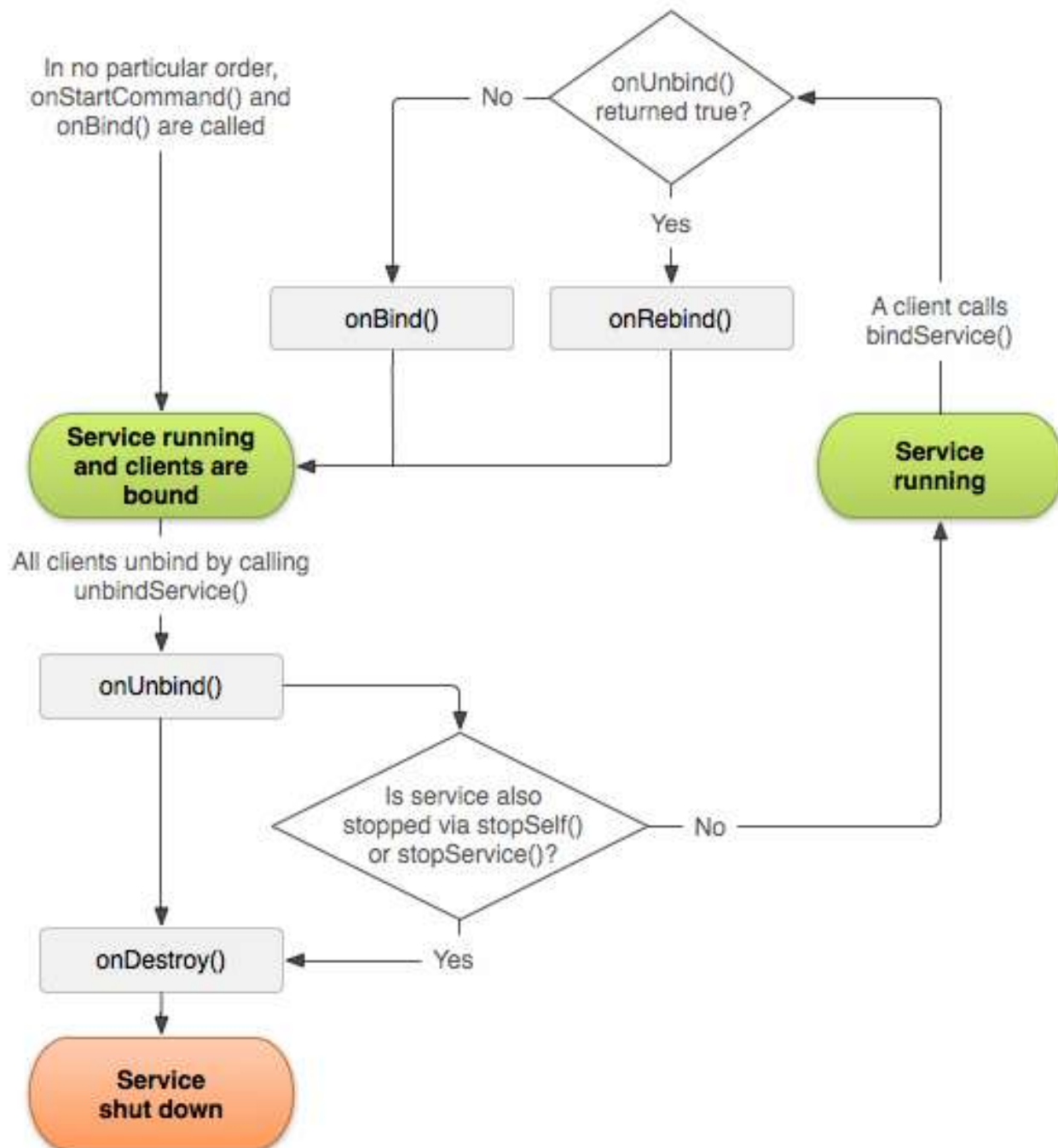
1.3 Service生命周期

- `onBind()`：当另一个组件通过调用 `bindService()` 与服务绑定时，系统将调用此方法。
- `onUnbind()`：当另一个组件通过调用 `unbindService()` 与服务解绑时，系统将调用此方法。



1.3 Service生命周期

- `onRebind()`：当旧组件与服务解绑后，另一个新组件与服务绑定，`onUnbind()` 返回 `true` 时，系统将调用此方法。



2 Android多线程编程

2.1 Android线程

- 线程：
 - 独立的程序单元
 - 多个线程可以并行工作
 - 在多处理器系统中，每个中央处理器（CPU）单独运行一个线程
 - 在单处理器系统中，处理器会给每个线程一小段时间，在这个时间段内线程被执行，然后处理器在下一个时间段执行下一个线程，这样就产生了线程并行运行的假象
 - 无论线程是否真的并行工作，在宏观上可以认为子线程是独立于主线程的，且是能与主线程并行工作的程序单元

2.1 Android线程

- Android线程：
 - 主线程上耗时的处理过程会降低用户界面的响应速度，甚至导致用户界面失去响应
 - 当用户界面失去响应超过一定时间后，系统会允许用户强行关闭应用程序，并进行提示
 - 解决方法：将耗时的处理过程转移到子线程上
 - 缩短主线程的事件处理时间
 - 避免用户界面长时间失去响应

2.2 Android线程基本用法

```
1. class MyThread : Thread() {  
2.     override fun run() {  
3.         // 编写具体的逻辑  
4.     }  
5. }
```

```
1. MyThread().start()
```

```
1. Thread {  
2.     // 编写具体的逻辑  
3. }.start()
```

```
1. class MyThread : Runnable {  
2.     override fun run() {  
3.         // 编写具体的逻辑  
4.     }  
5. }
```

```
1. val myThread = MyThread()  
2. Thread(myThread).start()
```

```
1. thread {  
2.     // 编写具体的逻辑  
3. }
```

2.3 在子线程中更新UI

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {

    var text by remember { mutableStateOf("World") }

    Column(
        modifier = modifier,
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(text = "Hello $text!")
        Button(
            onClick = {
                thread {
                    Thread.sleep(1000) // 模拟耗时任务 (1秒)
                    text = "Android" // 直接修改text状态变量
                }
            }
        ) { Text(stringResource(R.string.button_change)) }
    }
}
```

CAndroidThreadTest

2.3 在子线程中更新UI

```
@Composable
fun Greeting(modifier: Modifier = Modifier) {

    var text by remember { mutableStateOf("World") }
    val handler = Handler(Looper.getMainLooper())

    Column(.....) {
        Text(text = "Hello $text!")
        Button(
            onClick = {
                thread {
                    Thread.sleep(1000) // 模拟耗时任务
                    handler.post {
                        text = "Android"
                    }
                }
            }
        ) { Text(stringResource(R.string.button_change)) }
    }
}
```

中国联通 | Club 3,79K/s

8:24

中国联通 | Club 32,08/s

8:25

Hello World!

Change Text

Hello Android!

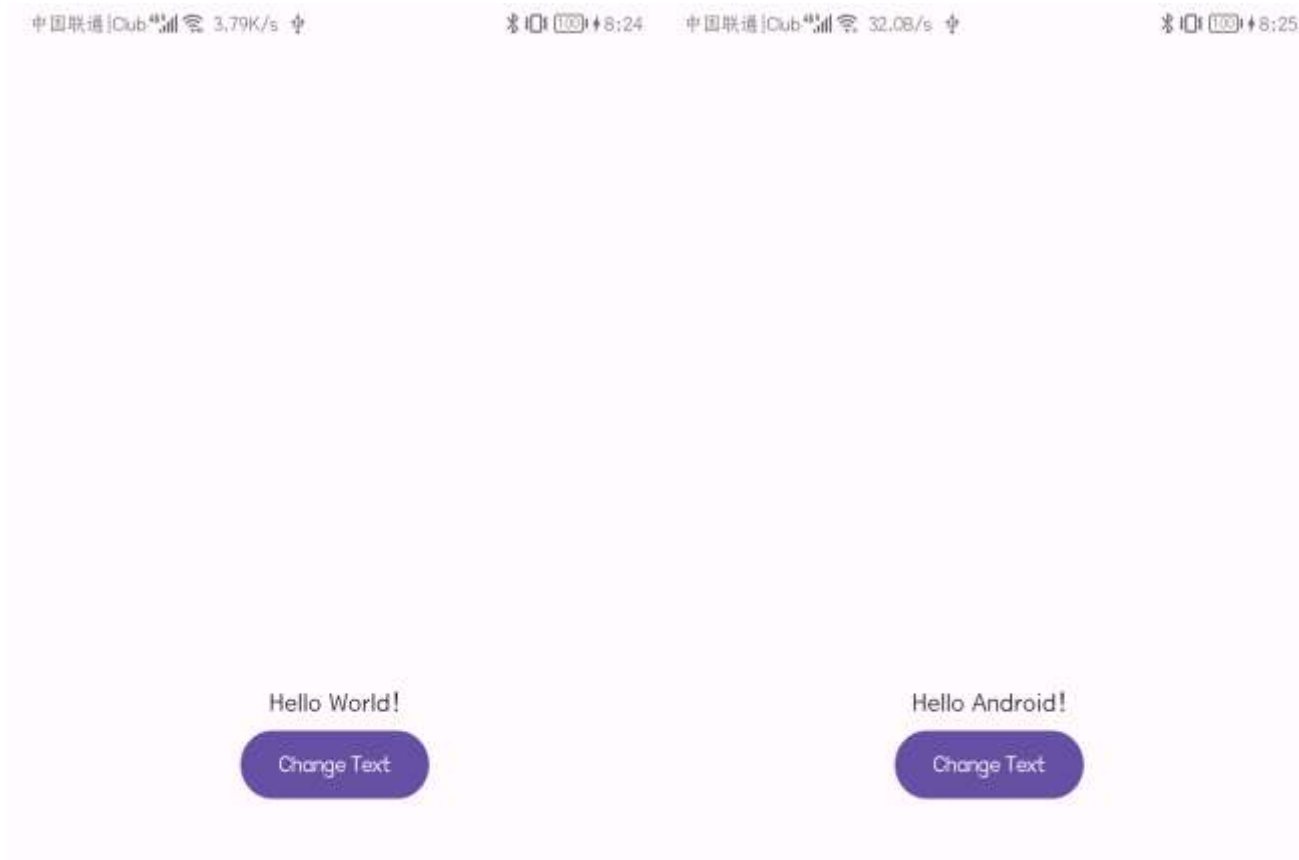
Change Text

2.3 在子线程中更新UI

```
@Composable
fun Greeting(modifier: Modifier = Modifier) {

    var text by remember { mutableStateOf("World") }
    val handler = object : Handler(Looper.getMainLooper()) {
        override fun handleMessage(msg: Message) {
            if (msg.what == 1) { text = "Android" }
        }
    }

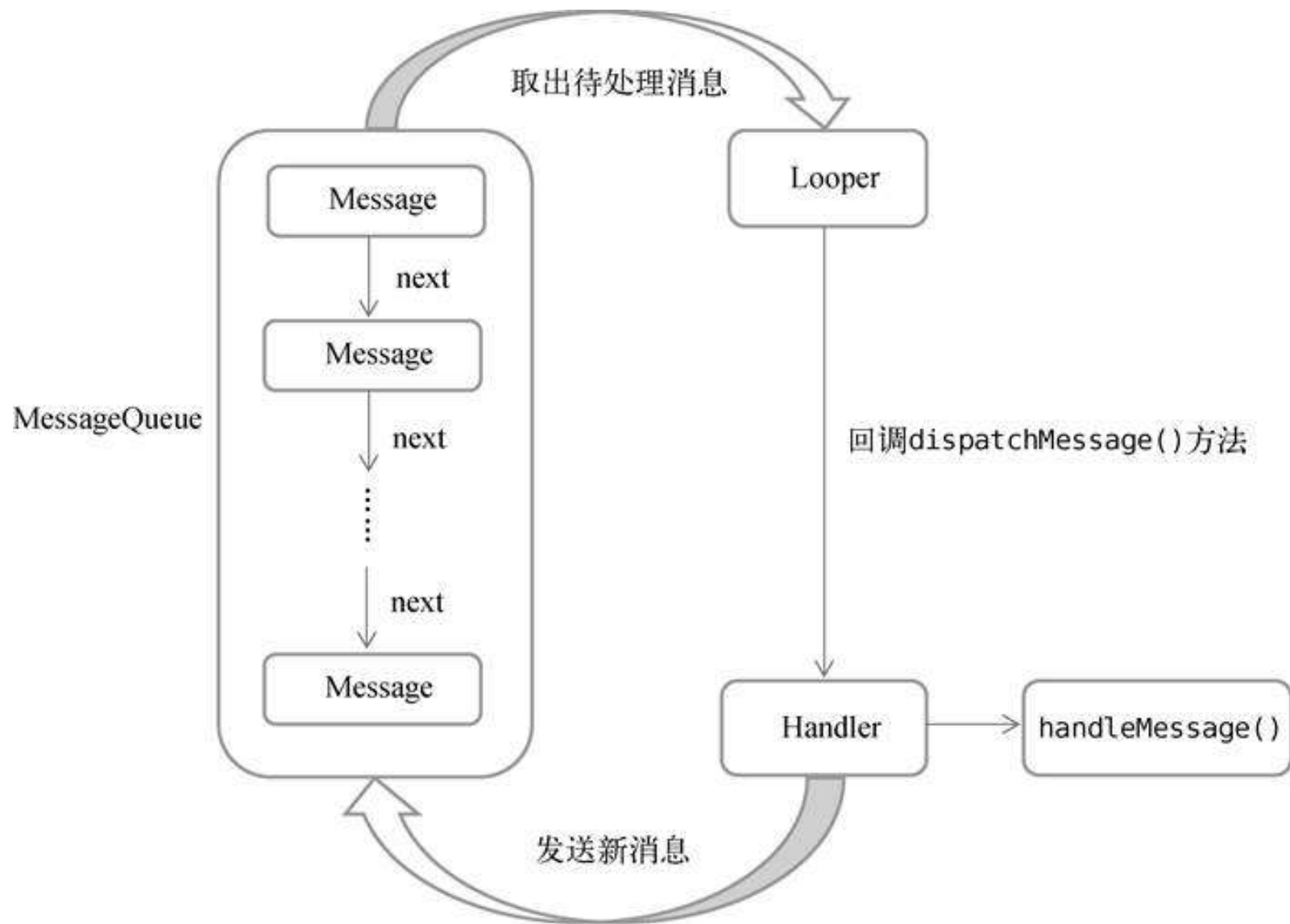
    Column(.....) {
        Text(text = "Hello $text!")
        Button(
            onClick = {
                thread {
                    Thread.sleep(1000) // 模拟耗时任务
                    handler.sendMessage(1)
                }
            }
        ) { Text(stringResource(R.string.button_change)) }
    }
}
```



2.4 异步消息处理机制

- Android线程异步消息处理机制解析（4个部分）：
 - Message：在线程之间传递的消息，可以在内部携带少量的信息，用于在不同线程之间传递数据。
 - Handler：主要用于发送和处理消息，发送消息一般是使用Handler的sendMessage()方法、post()方法等，发出的消息经过一系列处理后会最终传递到Handler的handleMessage()方法。
 - MessageQueue：消息队列，主要用于存放所有通过Handler发送的消息，这部分消息会一直存在于消息队列中等待被处理，每个线程中只会有一个MessageQueue对象。
 - Looper：每个线程中MessageQueue的管理者，调用Looper的loop()方法后会进入无限循环，每当发现MessageQueue中存在一条消息时就会将它取出并传递到Handler的handleMessage()方法，每个线程只有一个Looper对象。

2.4 异步消息处理机制



2.5 协程

- 协程 (Coroutines)：处理异步编程的轻量级框架
 - 协程本质上是轻量级线程
 - 用顺序代码方式写异步操作
 - 避免回调地狱
 - 使用suspend函数处理耗时操作

2.5 协程

```
@Composable
fun CounterScreen() {
    var count by remember { mutableStateOf(0) }
    var isRunning by remember { mutableStateOf(false) }

    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(text = "Count: $count")

        Button(onClick = { isRunning = !isRunning }) {
            Text(if (isRunning) "Stop" else "Start")
        }

        LaunchedEffect(isRunning) {
            while (isRunning) {
                delay(1000)
                count++
            }
        }
    }
}
```

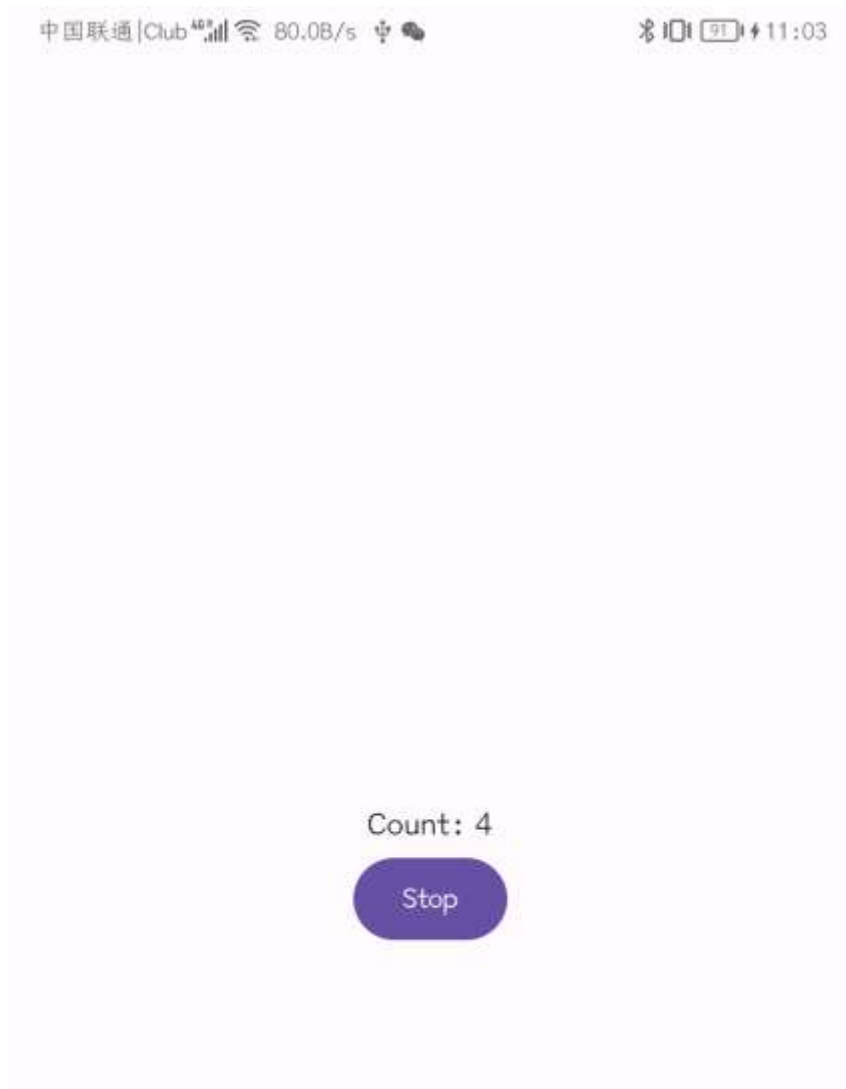
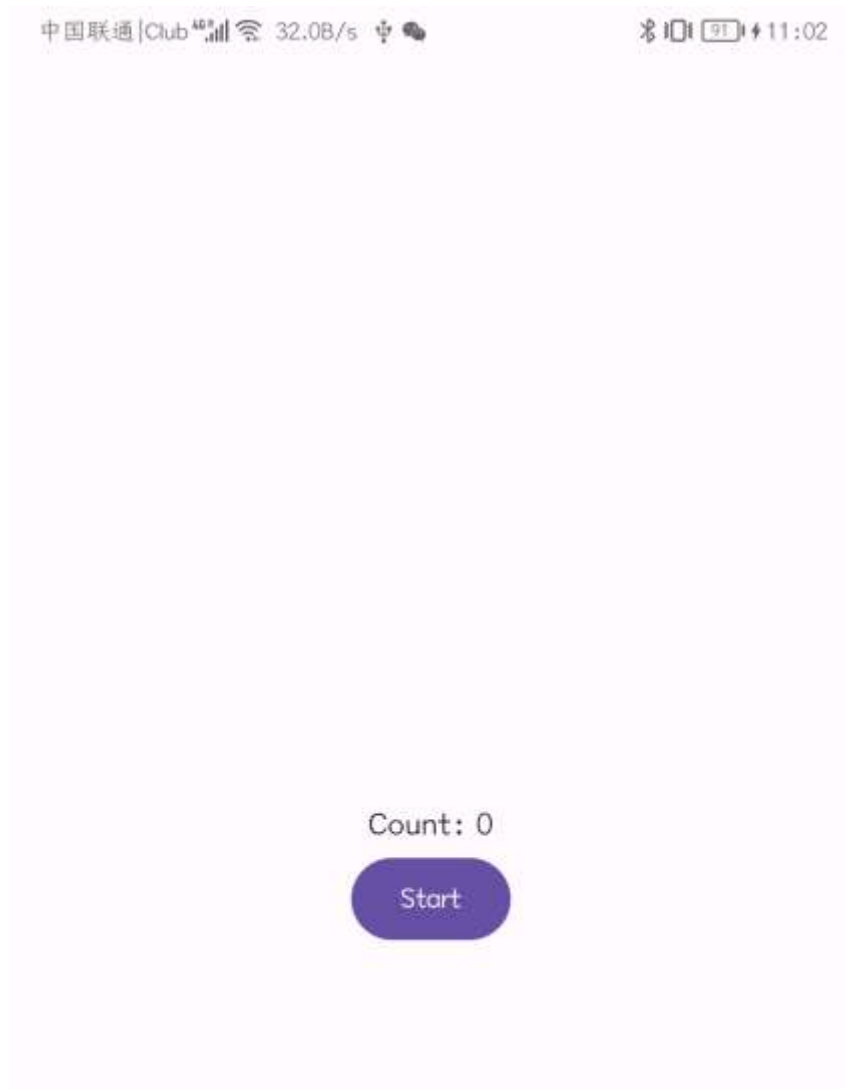
// 简单的计数器示例

```
@Composable
fun CounterScreen() {
    var count by remember { mutableStateOf(0) }
    var isRunning by remember { mutableStateOf(false) }
    val scope = rememberCoroutineScope()

    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(text = "Count: $count")

        Button(onClick = {
            isRunning = !isRunning
            if (isRunning) {
                scope.launch {
                    while (isRunning) {
                        delay(1000) // 每秒增加1
                        count++
                    }
                }
            }
        }) {
            Text(if (isRunning) "Stop" else "Start")
        }
    }
}
```

2.5 协程



3 Service基本用法

3.1 启动方式-定义Service

 New Android Component ✕

Service

Creates a new service component and adds it to your Android manifest

Class Name

MyService

☐ Exported

☒ Enabled

Source Language

Kotlin

```
class MyService : Service() {  
  
    override fun onBind(intent: Intent): IBinder {  
        TODO( reason: "Return the communication channel to the service.")  
    }  
}
```

Cancel Finish

3.1 启动方式-定义Service

```
class MyService : Service() {  
  
    .....  
  
    override fun onCreate() {  
        super.onCreate()  
    }  
  
    override fun onStartCommand(intent: Intent, flags: Int, startId: Int): Int {  
        return super.onStartCommand(intent, flags, startId)  
    }  
  
    override fun onDestroy() {  
        super.onDestroy()  
    }  
  
}
```

```
<service  
    android:name=".MyService"  
    android:enabled="true"  
    android:exported="false"/>
```

3.2 启动方式-启动和停止Service

```
@Composable
fun Greeting(modifier: Modifier = Modifier) {
    val context = LocalContext.current

    Column(...){
        Button(
            onClick = {
                val intent = Intent(context, MyService::class.java)
                context.startService(intent) // 启动Service
            },
            modifier = Modifier.fillMaxWidth(0.4f)
        ) {
            Text(stringResource(R.string.button_start))
        }

        Spacer(modifier = Modifier.height(32.dp))

        Button(
            onClick = {
                val intent = Intent(context, MyService::class.java)
                context.stopService(intent) // 停止Service
            },
            modifier = Modifier.fillMaxWidth(0.4f)
        ) {
            Text(stringResource(R.string.button_stop))
        }
    }
}
```

```
class MyService : Service() {

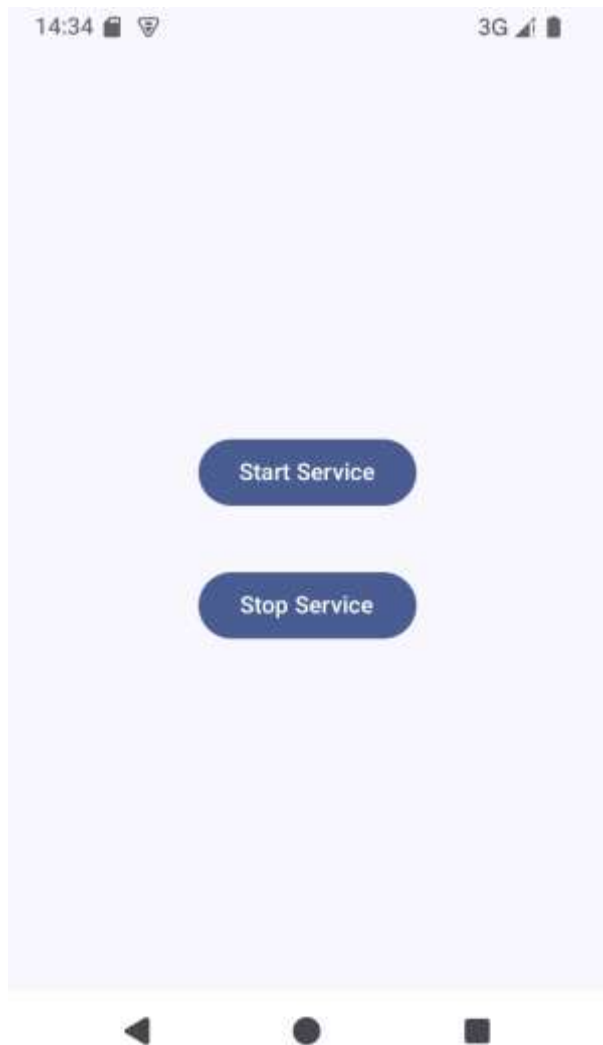
    .....

    override fun onCreate() {
        super.onCreate()
        Log.d("MyService", "onCreate executed")
    }

    override fun onStartCommand(intent: Intent,
        flags: Int, startId: Int): Int {
        Log.d("MyService", "onStartCommand executed")
        return super.onStartCommand(intent, flags, startId)
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d("MyService", "onDestroy executed")
    }
}
```

3.2 启动方式-启动和停止Service



```
MyService      com.example.cservicetest    D  onCreate executed
MyService      com.example.cservicetest    D  onStartCommand executed
```

```
MyService      com.example.cservicetest    D  onCreate executed
MyService      com.example.cservicetest    D  onStartCommand executed
MyService      com.example.cservicetest    D  onDestroy executed
```

3.3 绑定方式-Activity和Service通信

```
class MyService : Service() {  
  
    private val mBinder = DownloadBinder()  
  
    inner class DownloadBinder : Binder() {  
  
        fun startDownload() {  
            Log.d("MyService", "startDownload executed")  
        }  
  
        fun getProgress(): Int {  
            Log.d("MyService", "getProgress executed")  
            return 0  
        }  
  
    }  
  
    override fun onBind(intent: Intent): IBinder {  
        return mBinder  
    }  
  
    .....  
}
```

```
@Composable  
fun Greeting(modifier: Modifier = Modifier,  
    connection: ServiceConnection) {  
    val context = LocalContext.current  
  
    Column(.....){  
        Button( // 启动Service  
            onClick = { myStartService(context) },  
            modifier = Modifier.fillMaxWidth(0.4f)  
        ) { Text(stringResource(R.string.button_start)) }  
  
        Button( // 停止Service  
            onClick = { myStopService(context) },  
            modifier = Modifier.fillMaxWidth(0.4f)  
        ) { Text(stringResource(R.string.button_stop)) }  
  
        Button( // 绑定Service  
            onClick = { myBindService (context, connection) },  
            modifier = Modifier.fillMaxWidth(0.4f)  
        ) { Text(stringResource(R.string.button_bind)) }  
  
        Button( // 解绑Service  
            onClick = { myUnbindService (context, connection) },  
            modifier = Modifier.fillMaxWidth(0.4f)  
        ) { Text(stringResource(R.string.button_unbind)) }  
    }  
}
```

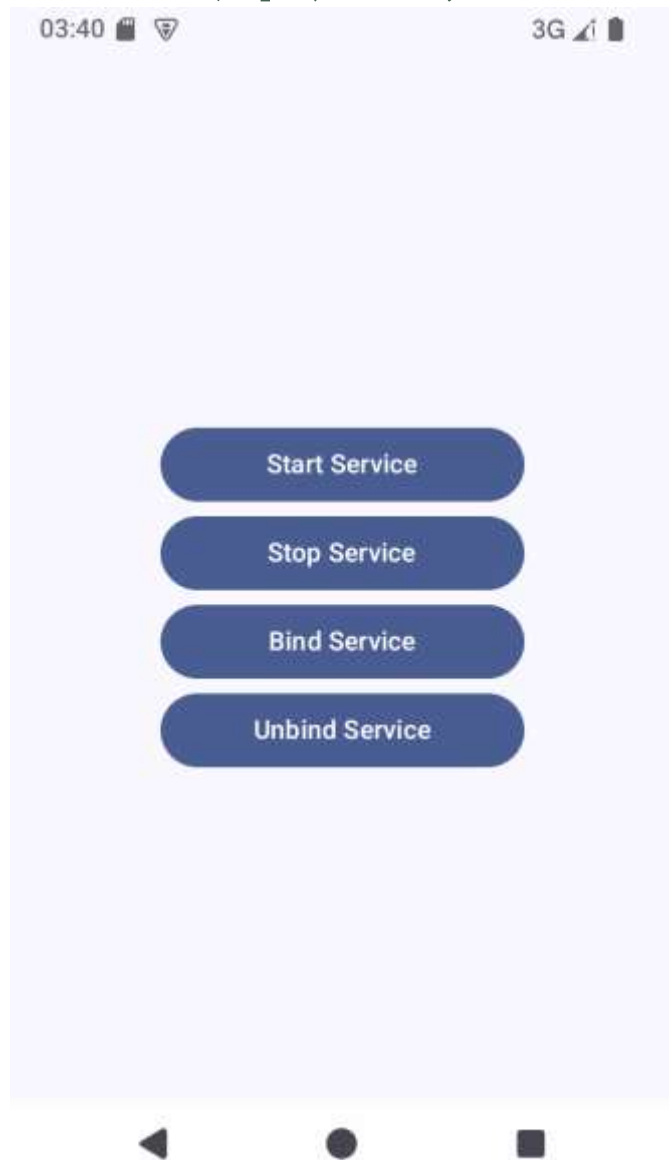
3.3 绑定方式-Activity和服务通信

```
class MainActivity : ComponentActivity() {  
  
    private var downloadBinder: MyService.DownloadBinder? = null  
    private val connection = object : ServiceConnection {  
        override fun onServiceConnected(name: ComponentName, service: IBinder) {  
            downloadBinder = service as MyService.DownloadBinder  
            downloadBinder?.startDownload()  
            downloadBinder?.getProgress()  
        }  
        override fun onServiceDisconnected(name: ComponentName) {  
            downloadBinder = null  
        }  
    }  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        .....  
        setContent {  
            .....  
            Greeting(.....,  
                connection)  
            .....  
        }  
        .....  
    }  
}
```

3.3 绑定方式-Activity和服务通信

```
private fun myStartService (context: Context) {  
    val intent = Intent(context, MyService::class.java)  
    context.startService(intent)  
}  
  
private fun myStopService (context: Context) {  
    val intent = Intent(context, MyService::class.java)  
    context.stopService(intent)  
}  
  
private fun myBindService (context: Context, connection: ServiceConnection) {  
    val intent = Intent(context, MyService::class.java)  
    context.bindService(intent, connection, Context.BIND_AUTO_CREATE)  
}  
  
private fun myUnbindService (context: Context, connection: ServiceConnection) {  
    try {  
        context.unbindService(connection)  
    } catch (e: IllegalArgumentException) {  
        e.printStackTrace()  
    }  
}
```


3.3 绑定方式-Activity和服务通信



```
MyService      com.example.cservicetest  D  onCreate executed
MyService      com.example.cservicetest  D  startDownload executed
MyService      com.example.cservicetest  D  getProgress executed
```

```
MyService      com.example.cservicetest  D  onCreate executed
MyService      com.example.cservicetest  D  startDownload executed
MyService      com.example.cservicetest  D  getProgress executed
MyService      com.example.cservicetest  D  onDestroy executed
```


4 前台Service

4 前台Service

```
class MainActivity : ComponentActivity() {
    .....
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

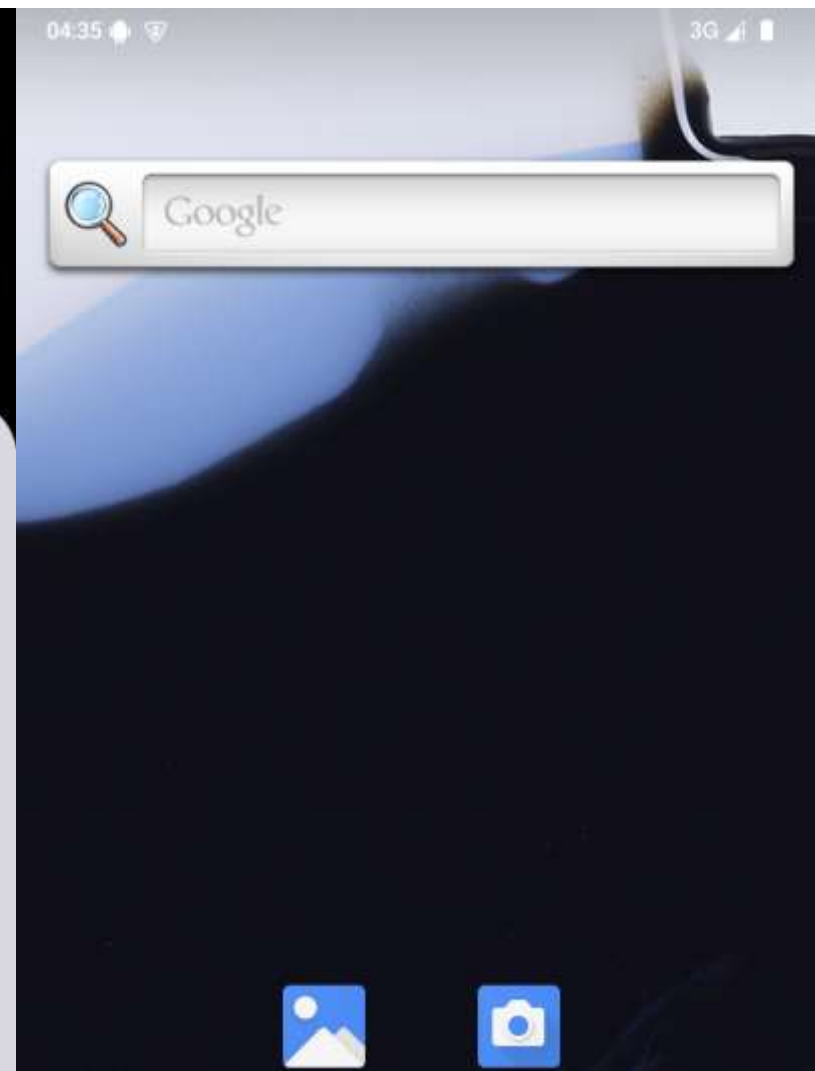
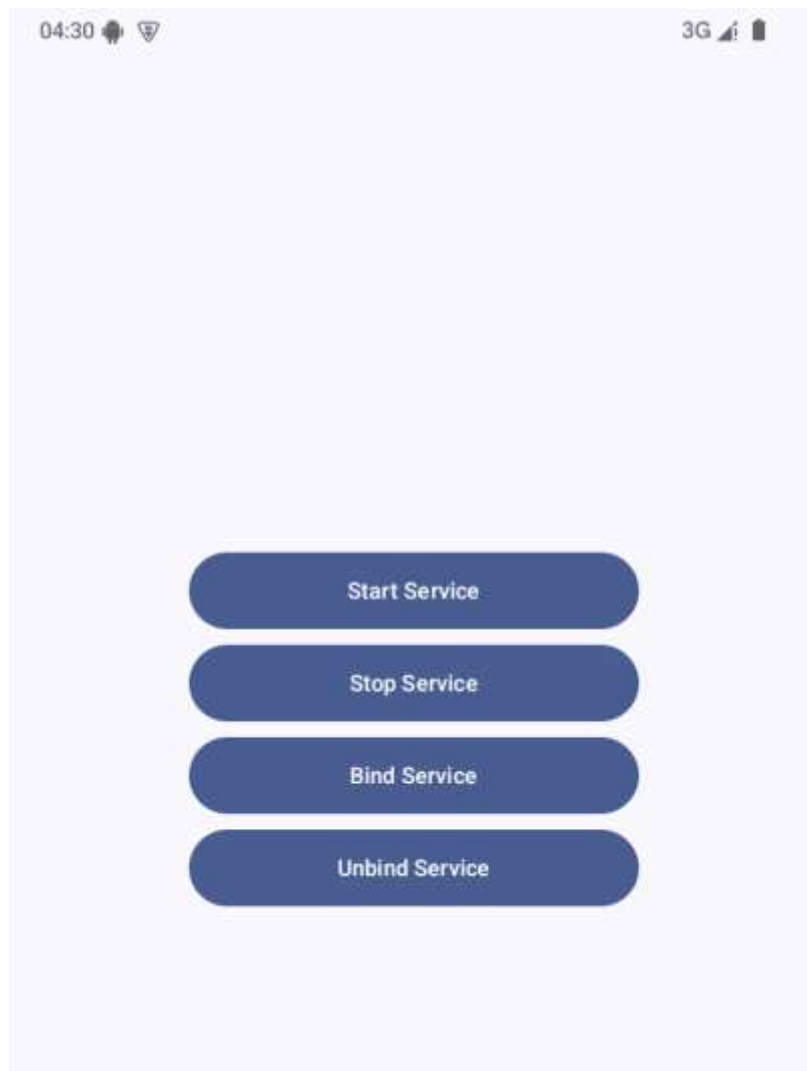
        val manager = getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel("my_service", "前台Service通知",
                NotificationManager.IMPORTANCE_DEFAULT)
            manager.createNotificationChannel(channel)
        }
        .....
    }
    .....
}
```

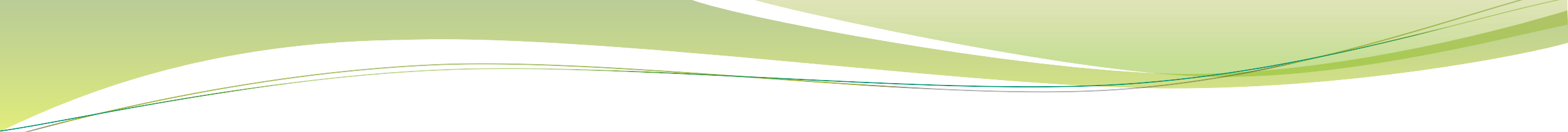
```
class MyService : Service() {
    .....
    override fun onCreate() {
        .....
        val intent = Intent(this, MainActivity::class.java)
        val pi = PendingIntent.getActivity(this, 0, intent, PendingIntent.FLAG_IMMUTABLE)
        val notification = NotificationCompat.Builder(this, "my_service")
            .....
            .setContentIntent(pi)
            .build()
        startForeground(1, notification)
    }
    .....
}
```

```
<uses-permission android:name=
    "android.permission.POST_NOTIFICATIONS"/>
<uses-permission android:name=
    "android.permission.FOREGROUND_SERVICE"/>
<uses-permission android:name=
    "android.permission.FOREGROUND_SERVICE_DATA_SYNC"/>
```

```
.....
<service
    android:name=".MyService"
    android:foregroundServiceType="dataSync"
    android:enabled="true"
    android:exported="false"/>
.....
```

4 前台Service





感谢观看！

